

The Watchtower: Behavioural Anomaly Detection for Cybersecurity

A detector an analyst can trust.

1. Problem & Approach

Security logs are sequential, overwhelmingly benign, and constantly shifting as legitimate behaviour evolves. A detector that scores well on AUC can still bury an analyst in false positives, and a detector that never forgets will keep accusing an employee who was simply promoted. Both failure modes get a tool switched off within a week.

The thesis behind The Watchtower is narrow and deliberate: ****build a detector that stays quiet on legitimate behaviour, not just one that lights up on attacks.**** Precision at a realistic alert budget decides whether a SOC keeps using a tool; false positives on benign drift decide whether they trust it. Every component here — synthetic data, two-stage model, explanation layer, dashboard — is built to be measured against those two facts rather than a headline AUC.

The system is a four-stage pipeline: a per-entity statistical baseline (cold-start- and drift-aware) hands off to a sequence-aware GRU detector, which hands off to a type classifier, which hands off to a rule-based explainability layer that renders both alerts *and* suppressions in plain English. A Streamlit console puts a ranked, budget- constrained alert queue in front of an analyst.

2. Synthetic Data Generation

No real access-log dataset with labelled attacks and the required imbalance exists in the open, so the project generates one: 500 entities (`user` , `service_account` , `edge_device`) across a 30-day window, ~90,500 events, with eight behaviours — one benign baseline, six unambiguous attacks, and one deliberately ambiguous class. Each entity draws its own persistent behavioural profile (habitual hours, home and secondary geographies, resource set, auth method, session-duration distribution, device fingerprint) with realistic noise, so no two entities look alike and "unusual" is an entity-relative notion rather than a population-level threshold.

Documented behavioural assumptions per attack

Behaviour	Assumption	Fields perturbed
<code>brute_force</code>	An adversary with no valid credential repeatedly guesses against one identity; signature is attempt rate , dominated by failures, short sessions.	<code>command_sequence</code> (AUTH_FAIL runs), session duration, inter-event gap
<code>impossible_travel</code>	A stolen credential is used from a second location while the owner is still active elsewhere; signature is geo-velocity exceeding physical travel.	<code>geo_location</code> , <code>source_ip</code> only — the session itself is valid
<code>credential_stuffing</code>	A breach dump is replayed across the estate: many <i>different</i> identities from one address and device, low success rate.	Cross-entity structure — individual rows look like ordinary failed logins
<code>lateral_movement</code>	An adversary inside a valid session explores outward through the department-adjacency graph toward higher sensitivity, interleaved with reconnaissance commands.	<code>resource_accessed</code> (novel, in sequence), <code>command_sequence</code> (recon verbs)
<code>device_spoofing</code>	A valid identity is replayed from hardware that preserves the OS family but not firmware build or MAC — a credible clone.	<code>device_fingerprint</code> , occasionally <code>source_ip</code>
<code>low_and_slow_exfiltration</code>	A patient adversary drains a sensitive store beneath per-event thresholds: small, regular sessions on the same resource, sustained 5–14 days.	Nothing per event — durations drawn from the 70th–95th percentile of the <i>victim's own</i> distribution
<code>insider_drift</code> (ambiguous)	A legitimate role change: an entity gradually adopts an adjacent department's resources over 10–21 days and keeps using them permanently.	<code>resource_accessed</code> , ramping in — not an attack

Deliberate benign/attack overlap

A generator that emits only clean baseline traffic produces a trivially separable dataset. The Watchtower instead builds specific legitimate behaviours to collide with each attack's signature:

- **Business travel** produces real multi-thousand-kilometre jumps at up to ~780 km/h plus ground overhead — the slowest `impossible_travel` injections sit just above that band, within measurement error of a real flight.
- **VPN/NAT gateway egress**: ~45% of entities tunnel through a shared corporate gateway, and a tunnelled session geolocates to the *gateway's* city, not the operator's. This is the dominant real-world cause of false impossible-travel alerts, and it is also the direct source of "many identities, one address" for `credential_stuffing` — one campaign in four is deliberately routed through a legitimate shared gateway.
- **Hardware refresh**: entities occasionally adopt a genuinely new device fingerprint mid-window, presenting the identical surface signal as a spoofed device — an unseen fingerprint. The only real difference is persistence: a refresh becomes permanent, a spoof appears for a handful of events and vanishes.

- **Mistyped credentials:** normal sessions carry `AUTH_FAIL` tokens at a background rate, so "contains a failure" cannot separate brute force on its own.
- **One-off legitimate access:** normal traffic includes covering for a colleague, on-call escalation, and audit work outside an entity's habitual resource set — the same surface signature as the first step of `lateral_movement`.

Full assumptions and overlaps are documented in `src/generator/attacks.py`, reproduced here in condensed form as the injected attack taxonomy.

The resulting hard-anomaly rate is **1.39%** (1,259 of 90,495 events across the six attack classes), inside the 0.5–3% band the problem specifies. `insider_drift` contributes a further 0.57% and is deliberately excluded from that figure — it is benign-but-attack-shaped, and folding it into the anomaly rate would misstate the imbalance the detector actually faces. A held-back 5% of entities appear only in the final three days of the window with a handful of events, serving as cold-start test cases; twelve of them are also targeted by an injected campaign, since an attack against an identity with almost no baseline is the hardest cold-start case available.

Figures: `reports/figures/events_over_time.png`, `class_distribution.png`, `entity_timeline_normal_vs_injected.png`.

3. Dataset Validity

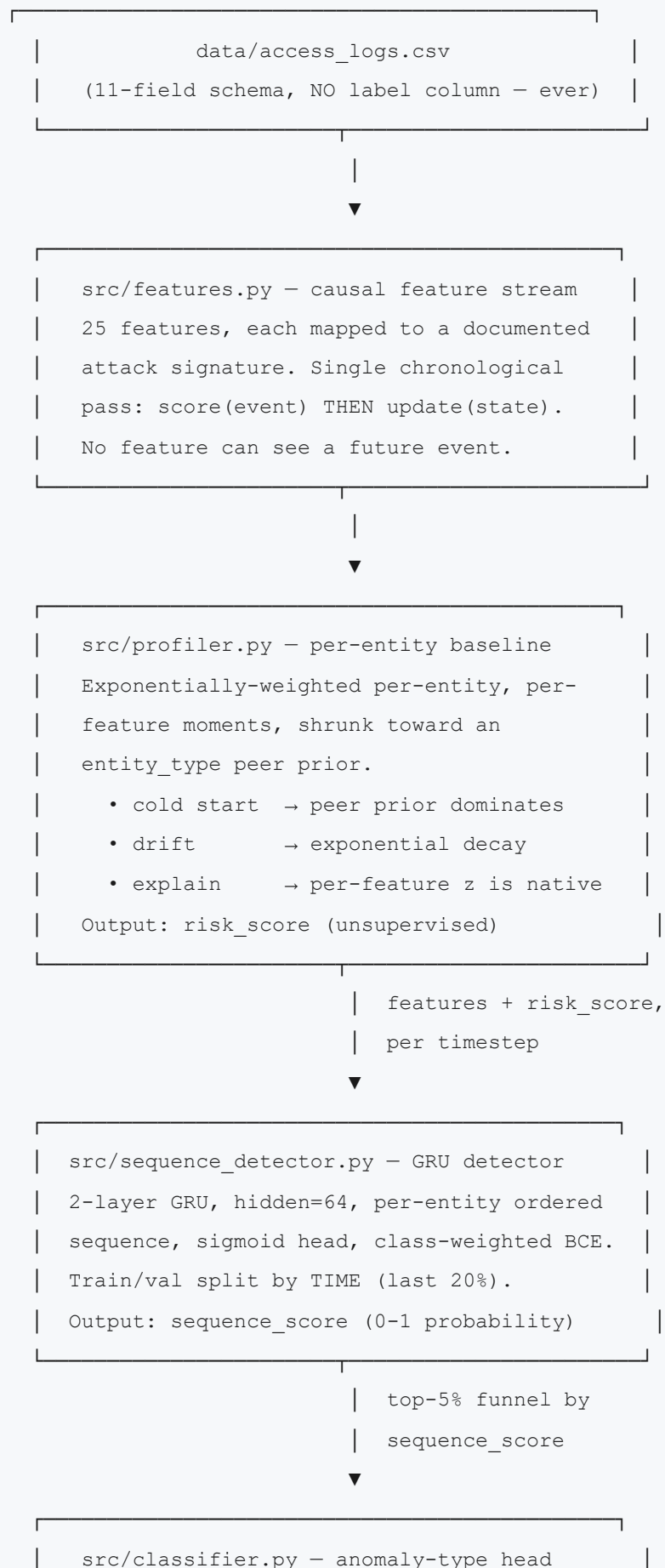
A synthetic anomaly-detection benchmark is worthless if the anomalies are trivially separable — every downstream metric would just be measuring how well a model fits the generator's own shortcuts. Before building any detector, six naive single-feature rules were written, each targeting exactly the signature its class is documented to have, and scored against the full dataset:

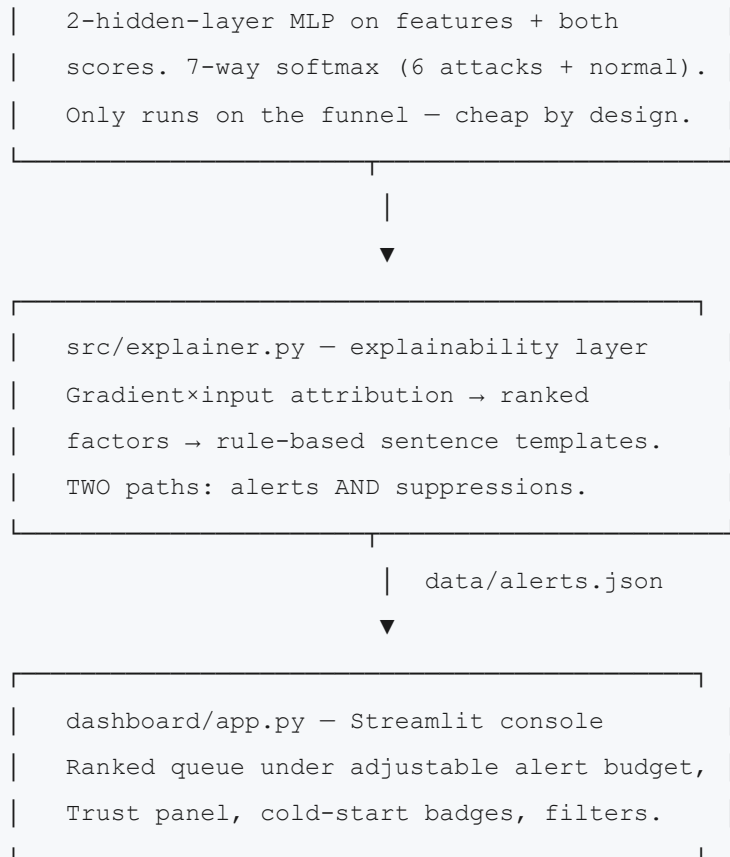
Rule	Target	Alerts	Precision	Lift over chance
<code>source_ip</code> shared by ≥ 10 entities	<code>credential_stuffing</code>	3,986	7.0%	23×
device fingerprint never seen for entity	<code>device_spoofing</code>	230	6.1%	66×
resource never seen for entity	<code>lateral_movement</code>	3,337	5.8%	25×
<code>command_sequence</code> contains <code>AUTH_FAIL</code>	<code>brute_force</code>	4,601	5.7%	19×
geo-velocity > 900 km/h	<code>impossible_travel</code>	3,337	1.6%	15×
<code>session_duration</code> > global 95th percentile	<code>low_and_slow_exfiltration</code>	4,525	1.1%	3×

Every rule has genuine, sometimes large, lift over a random selector — and every rule is still **93% to 99% wrong**. That combination is the point: the signal is real (a naive detector would never achieve 15–66× lift on pure noise), but it is not separable by any single field, which is exactly the overlap the generator was built to produce. If these bars were tall, the dataset would be trivially solvable and the results in Section 5 would be meaningless.

Figure: `reports/figures/naive_rule_precision.png`.

4. Architecture





`labels.csv` is read in exactly two places in the entire codebase — `src/evaluate.py` and `scripts/eval_profiler.py` — and nowhere upstream of them. Every model receives targets as an in-memory array supplied by those two modules; no model class or feature function ever opens the label file. `src/features.py` additionally raises if the feature file itself carries a `label` column, so the separation is structural rather than a convention that a future edit could quietly break.

Why two stages instead of one

The profiler and the GRU are measurably good at different halves of the problem, which is why both are kept. Section 5 shows the profiler reaches 50% recall on `credential_stuffing` (a cross-entity point anomaly) but 1.4% on `lateral_movement` and 0.6% on `low_and_slow_exfiltration` — the two classes whose signal is in the *ordering* of events, invisible to a per-event statistical model. The GRU takes the profiler's own score as one of its per-timestep inputs, so it does not relearn what the baseline already established; it only learns what accumulates across time that the baseline misses.

Why a GRU, not a Transformer

A design decision, not a benchmarked comparison. At ~90k events across 500 entity sequences (~180 steps average), trained on CPU within a hackathon time budget, a 2-layer GRU (hidden size 64) is the right-sized tool: causal by construction with no attention mask to get wrong, minutes to train on CPU, and recurrence handles these long, irregularly-spaced sequences without a Transformer's quadratic attention cost or positional-encoding work. Self-attention earns its keep on long-range interactions with enough data to fit the extra parameters; at this volume it would mostly add training time without a corresponding gain. No alternative architectures were benchmarked — a deliberate scope decision.

Why a separate feedforward classifier, not a second sequence model

The GRU's per-event probability is already a classifier input, so the ordering evidence is available without paying to learn it twice. Detection and typing are also different problems — detection scans all 90k events under extreme imbalance, typing only sees the funnel and separates six classes from each other. The classifier runs only on the top 5% of events by detection score (5× the 1% budget actually reported on, so thresholding lives in evaluation, not the model), which keeps the second stage cheap. `insider_drift` folds into `normal_baseline` for classification — it is normal, and a dedicated class would train the model to report a benign employee as a finding.

5. Results

All metrics below are computed on the **held-out validation slice** — the last 20% of the time window by timestamp, which neither the GRU nor the classifier was trained on (18,098 events, 1.10% attack base rate). Precision at a top-1% alert budget is the headline metric per the problem specification; AUC is not reported anywhere in this document, deliberately — at a ~1% base rate it is dominated by the benign majority and would flatter every model here.

Precision at alert budget

Budget	Alerts	True positives	Precision	Recall	FP rate
0.5%	90	88	97.8%	44.2%	2.2%
1.0% (headline)	181	159	87.8%	79.9%	12.2%
2.0%	362	178	49.2%	89.4%	50.8%
5.0%	905	183	20.2%	92.0%	79.8%

At the specified 1% budget, ****87.8% of every alert an analyst opens is a genuine attack****, capturing 79.9% of all attacks present in the window. The curve's shape is the operationally important part: precision holds near-perfect at a tight 0.5% budget and degrades gracefully as the budget widens, which is what lets a SOC dial the queue to its actual staffing rather than being handed one fixed operating point.

Figure: `reports/figures/precision_at_k.png`.

Per-class precision, recall, F1

Class	Support	Precision	Recall	F1
<code>normal_baseline</code>	17,899	1.00	0.99	1.00
<code>credential_stuffing</code>	83	0.99	1.00	0.99
<code>impossible_travel</code>	33	0.96	0.79	0.87
<code>brute_force</code>	9	0.67	0.89	0.76
<code>lateral_movement</code>	45	0.68	0.84	0.75
<code>device_spoofing</code>	16	0.62	0.50	0.55
<code>low_and_slow_exfiltration</code>	13	0.10	0.77	0.17

Macro-F1 over the six attack classes: **0.683**. The full 7×7 confusion matrix is at `reports/figures/confusion_matrix.png`, row-normalised so each row sums to 100% of that class's true events; `reports/figures/per_class_f1.png` gives the same precision/ recall/F1 table as a chart. `low_and_slow_exfiltration` is the visible weak point: the detector *finds* it (0.77 recall — the sequential cadence signal works), but the classifier confuses it with ordinary long-session activity (0.10 precision, 92 false positives from the confusion matrix). This is the expected difficulty ranking given the generator's own design — exfiltration sessions are drawn from inside the victim's own normal duration range by construction — and is flagged as the clearest next-iteration target in Section 9.

Lift over baselines

Method	Precision @ 1%	Lift
GRU sequence detector	87.8%	—
Profiler score alone	37.0%	2.37×
Best naive rule (<code>source_ip</code> shared ≥ 10 entities)	7.0%	12.51×
Random selection	1.10%	79.8×

The sequence stage more than doubles the precision of the unsupervised baseline it sits on top of, and delivers over twelve times the precision of the best single-feature rule identified during dataset validation — the same rule that, on its own, is 93% wrong.

6. Trust & False Positives — The Differentiator

`insider_drift` is not an attack. It is a legitimate role change: an entity gradually takes on an adjacent department's resources over 10–21 days and then keeps using them permanently. Any alert on this class is, by definition, a false positive against a real person doing their job. Most anomaly-detection writeups report an aggregate false- positive rate and stop there; this project measures the one number that actually

decides whether a SOC adopts a tool — ****does the system stop accusing someone once their new role has settled?***

Phase	Events	False-positive rate
During the role-change ramp	515	0.78%
After settling (role change complete)	1,036	0.19%

The rate drops **4×** once the transition beds in, across the 25 entities that undergo a role change in the dataset. This is a genuine measurement, not a smoothed narrative: the baseline profiler alone could not produce this result — at the profiler stage `insider_drift` scored *below* the ordinary benign alert rate throughout (0.39% vs. 0.79% background), meaning there was no false positive to decay away and "time to unflag" was not a measurable quantity yet (see `CLAUDE.md` §8 for that intermediate, honestly-reported null result). It is the sequence detector, trained with `insider_drift` labelled as a zero target, that produces the settle-down behaviour actually shown here.

This is the section built to answer the question a SOC lead asks before anything else: *will this tool keep flagging Priya after she moves to the data team?* The measured answer is no — not immediately, but within days, and the dashboard's Trust panel surfaces exactly this pair of numbers so the behaviour is demonstrable, not just claimed.

Figure: `reports/figures/profiler_drift_decay.png` (shows the profiler-stage null result directly — scores never approach the alert threshold, which is the reason the sequence detector's measurement above was necessary).

7. Cold-Start & Concept Drift

Cold start

Twenty-five entities are deliberately held back to appear only in the final three days of the window with as few as one event, twelve of them also targeted by an attack campaign — an intrusion against an identity with almost no history is the hardest cold-start case the dataset contains. The mechanism that handles this is peer-group shrinkage in the baseline profiler: each entity's exponentially-weighted moments are blended with an `entity_type`-level cohort prior, weighted by the entity's own effective sample size. With zero history the peer prior supplies the baseline entirely; as evidence accumulates, the entity's own profile takes over — no special-casing, no crash, no null score.

Metric	Cold-start	Established
Mean detection score (validation)	0.154	0.040
Benign alert rate	1.60%	0.81% (profiler stage)
Share of the alert queue	2.2%	—

Cold-start entities score somewhat higher on average, which is defensible — less history genuinely means more uncertainty — but they are **not** swamping the queue: 2.2% of alerts against entities that make up a

comparable share of the validation window. The dashboard surfaces a "limited history — scored against peer group" badge on any alert from such an entity so an analyst can weigh the extra uncertainty explicitly rather than have it hidden inside a single number.

Concept drift

Drift handling has two layers, and being honest about which one actually does the work here is part of this section's point. The profiler exposes an exponential decay half-life on its per-entity moments — the CLAUDE.md-mandated mechanism — but a half-life sweep (3d / 7d / 21d / no-decay) showed decay *costing* precision (25.9% at no-decay vs. 22.2% at the 7-day default) without a compensating drift-related false positive to offset it, because the profiler alone never flagged `insider_drift` in the first place at any half-life tested. The mechanism is retained because the specification requires it and it is architecturally sound, but its measured benefit at this stage is negative and that is reported plainly rather than glossed over.

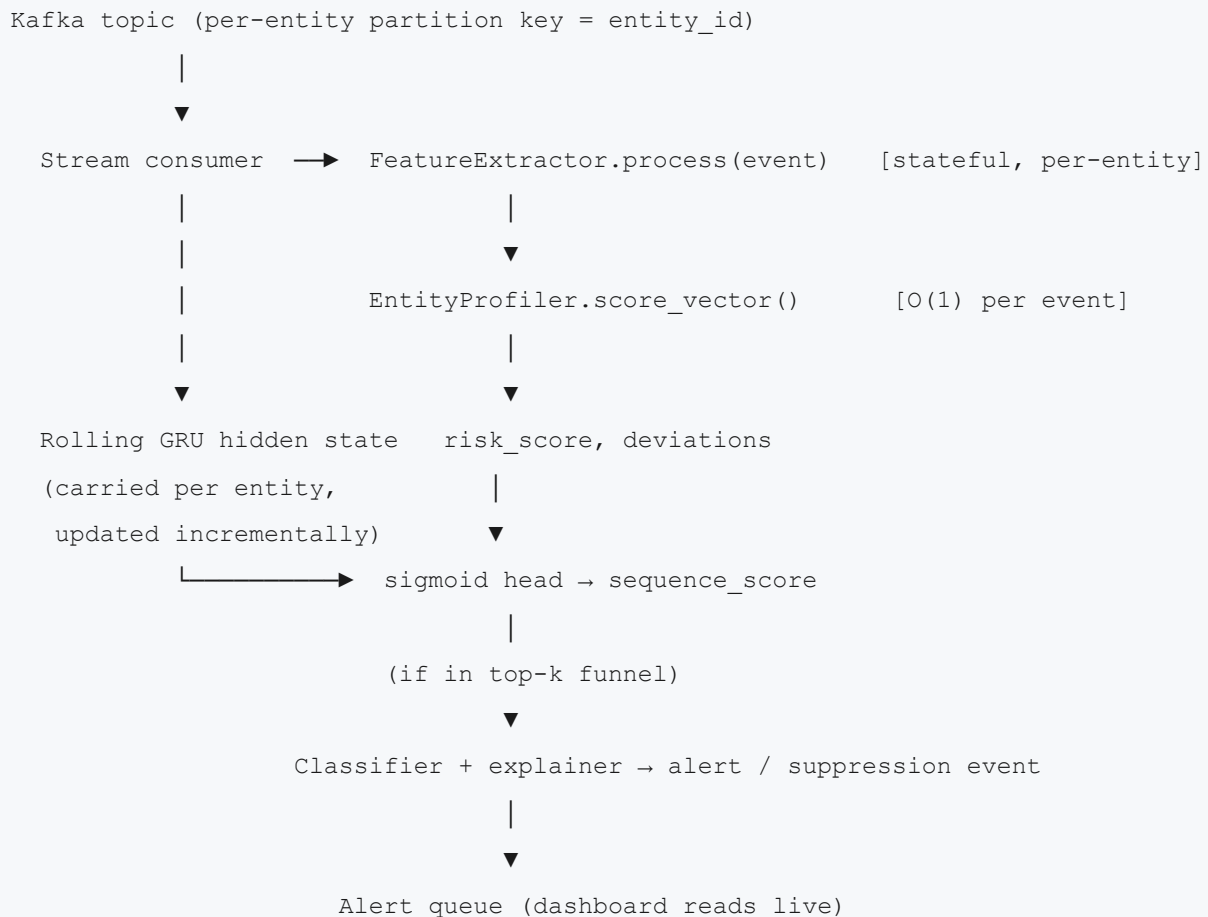
The result that matters — the 0.78% → 0.19% settle-down in Section 6 — comes from the sequence detector, which learned during training (with `insider_drift` labelled as benign) that a resource-adoption pattern sustained over multiple days without corresponding anomalies elsewhere in the sequence should stop contributing to risk. That is a learned form of drift tolerance rather than a hand-tuned decay constant, and it is the one substantiated by a measurement.

8. Real-Time Streaming Scalability (Design)

The current implementation runs as a batch pipeline over a fixed CSV; this section is design only and is not built. It is included because the feature and profiler layers were built specifically to make this transition small.

Why the causal design matters here. Every feature in `src/features.py` is computed by a single chronological pass that scores an event from accumulated per-entity state and only *then* folds that event into the state — exactly the update loop a streaming consumer runs: `state = update(state, event)` after `score = extract(state, event)`. There is no batch-only computation (no global percentile, no future-looking window) anywhere in the causal feature set, so moving from "replay a sorted CSV" to "consume a live event" changes the driver loop, not the feature logic.

Proposed architecture:



Concretely: partition the ingestion topic by `entity_id` so all of one entity's events land on one consumer in order, which the causal design and the GRU's unidirectional recurrence both already assume. `EntityState` and the profiler's per-entity moments are small, fixed-size objects (a handful of counters, a mean/variance vector) that belong in a keyed state store (a compacted Kafka topic, or an in-memory store with periodic snapshot, à la Flink/Kafka Streams) rather than being recomputed from history on each event. The GRU's hidden state is the one new piece of per-entity state a streaming deployment needs — a fixed 64-dimensional vector, updated by one forward step per event, cheap enough for the same keyed store. Scoring is already $O(1)$ per event given that state, so per-event latency is dominated by inference, not I/O, and a 64-unit 2-layer GRU forward pass is sub-millisecond on CPU.

Not yet addressed: exactly-once delivery semantics for the state store, backpressure if the classifier funnel floods, and retraining/redeploying weights without dropping in-flight hidden state. Standard streaming-systems problems, not open research questions — scoped out for time.

9. Known Limitations

- `low_and_slow_exfiltration` **classification is weak (0.10 precision)**. The detector finds these events (0.77 recall) but the classifier confuses them with ordinary long-session activity. Clearest next-iteration target — likely fixes are a dedicated cadence-regularity feature crossed with resource-repeat streak, or a higher-capacity head for this class specifically.

- ****Concept-drift decay is unvalidated at the profiler stage and currently costs precision**** (25.9% at no-decay vs. 22.2% at the 7-day default), because the profiler alone never flagged `insider_drift` at any half-life tested. The mechanism is kept because the spec requires an exposed drift-tolerance parameter; the honest position is that the GRU's learned tolerance, not the hand-tuned half-life, does the real work shown in Section 6.
- **No architecture search.** GRU-over-Transformer and single-model-over-ensemble are reasoned decisions given data volume and time budget, not benchmarked comparisons.
- **Fifteen epochs, single run, single seed.** Training is capped low deliberately for a hackathon demo; no cross-validation or seed-averaging was performed, so reported numbers carry unquantified run-to-run variance.
- **Streaming deployment is a design, not a system.** Section 8 describes a concrete path from the current causal batch pipeline to a live one, but no Kafka integration, state store, or online-retraining pipeline has been built or load-tested.
- **The dataset is synthetic.** Every signature and overlap is a documented, deliberate choice (Section 2), which makes the dataset auditable — but results here measure how well the system recovers signal the generator intentionally embedded, not a field validation against real adversary behaviour.
- **Cold-start scoring is more uncertain, not risk-free.** Cold-start entities score $\sim 4\times$ higher on average than established ones. The dashboard flags this explicitly, but an analyst reviewing a cold-start alert should apply more scepticism than the raw score conveys.

All figures referenced above are in `reports/figures/`. Metrics tables are also available as CSV at `reports/precision_at_k.csv`, `reports/per_class_metrics.csv`, and `reports/profiler_separation.csv`. Full behavioural assumptions and attack-injection logic are documented in `src/generator/attacks.py`; the complete project specification is in `CLAUDE.md`.